

Documentation

Introduction

PressE Pro is your all-in-one interaction system for Unity, letting you focus on creativity instead of complex coding. With this asset you can create responsive, player-friendly interactions in minutes, and not spend days with it. Whether you're shipping a small indie title or scaling a large production, PressE Pro delivers a cost-effective foundation that's easy to integrate and customize, saving your team time and resources.

Support

Email: faststudiosunity@gmail.com ✉

Forms: [Click Here](#) ↗

Jump right in

Getting Started

Learn how make the first steps

Interaction Types

Learn about all interaction types

Conditions

Learn how to add conditions to your interactions

Speed Up Workflow

Learn how to waste less time on setting things up

Components

Learn all useful components that comes with this asset

Scripting

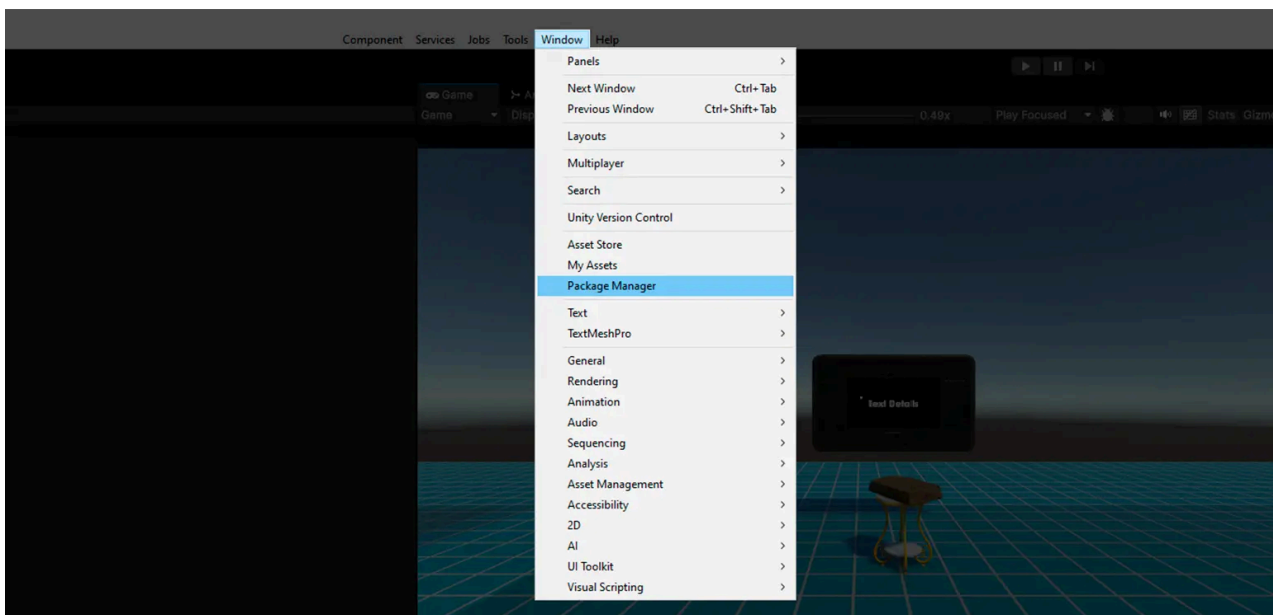
Learn how to do more complex interactions with scripting

Manual

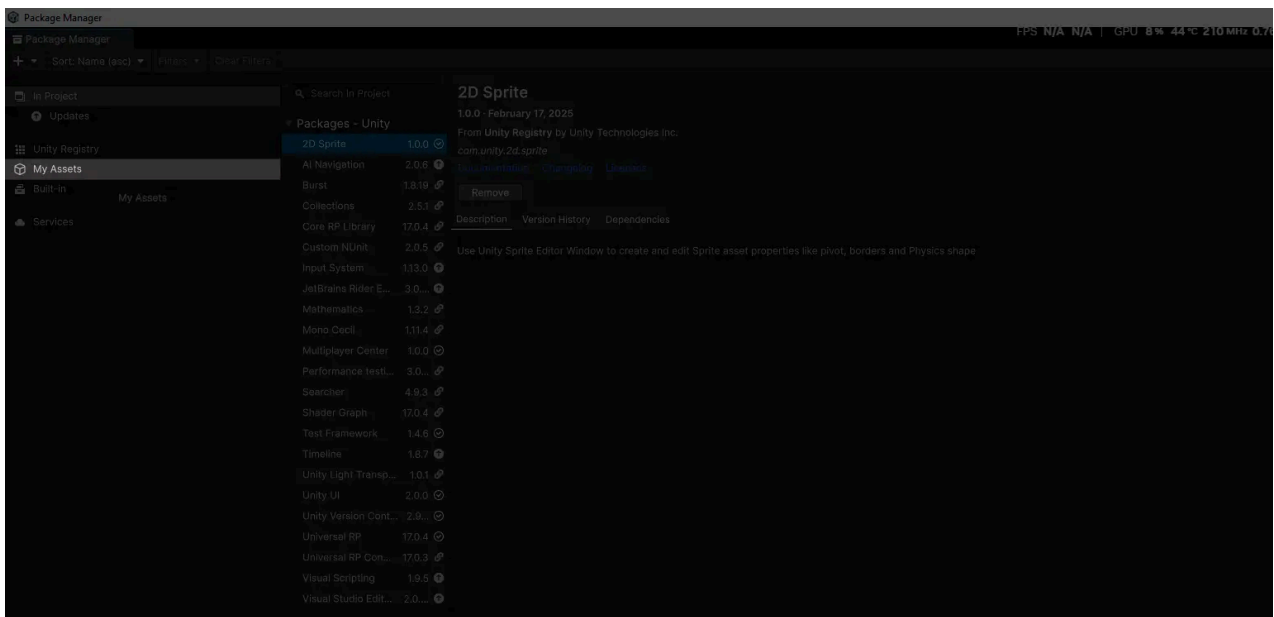
Getting Started

Installation

Step 1: In the **Unity Editor**, navigate to **Package Manager** (**Window > Package Manager**).



Step 2: In the **Package Manager**, click on **Packages** and select **My Assets** in the left side of the screen.



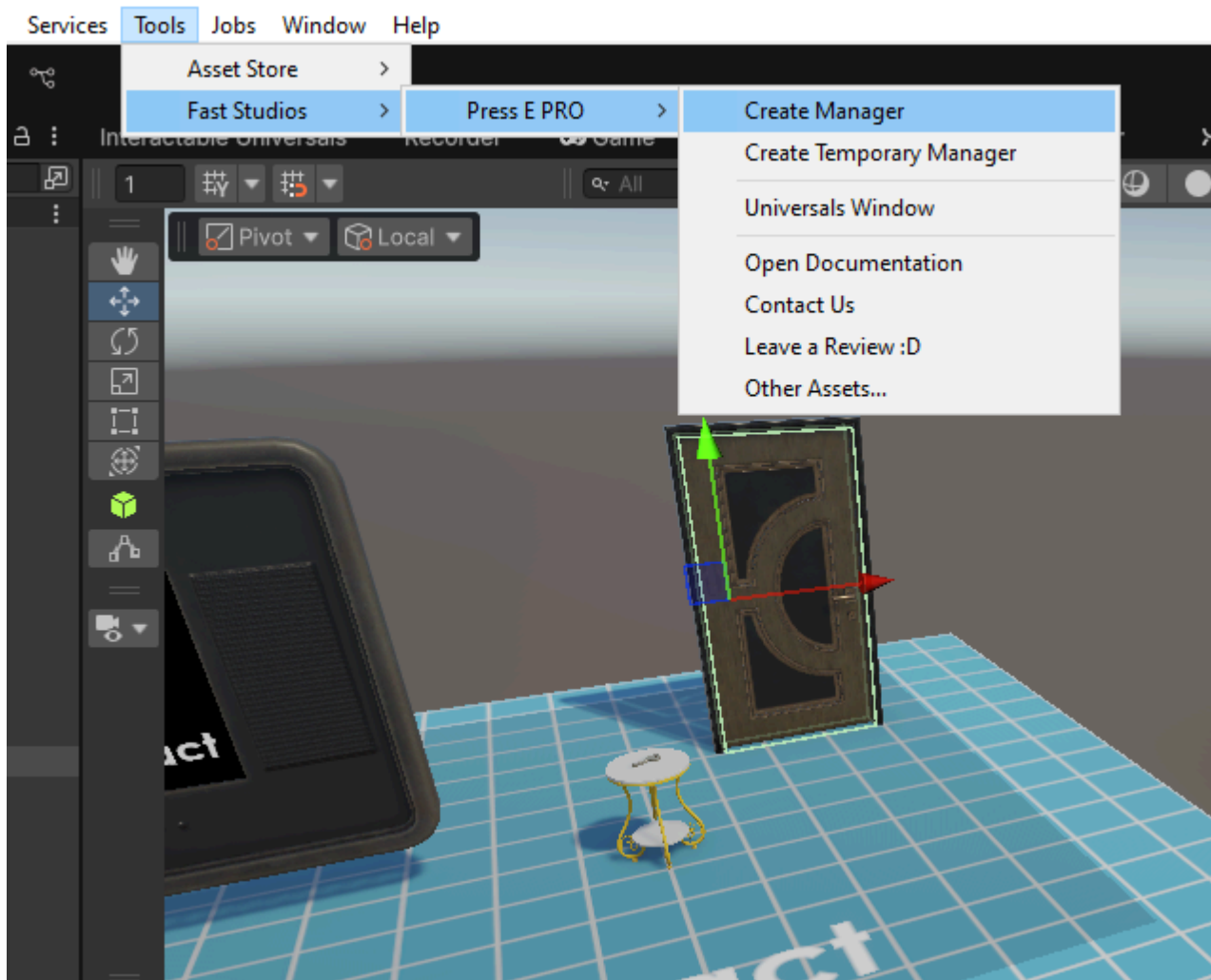
Step 3: From the list, choose **PressE PRO - A Complete Interaction System for Unity**

Step 4: In the bottom right corner, click **Download**

Step 5: And finally, click **Import**

Interaction Manager Setup

To create [Interaction Manager](#), go to **Tools > Fast Studios > Press E PRO > Create Interaction Manager**



- i** Remember to place the [Interaction Manager](#) in the **first scene** of your Build Settings. It's a **singleton** with `DontDestroyOnLoad`, which means there must be **only one** instance in your game. Because it persists across scene loads, adding it to the first scene keeps it active **from start to finish**. **Do not add another [Interaction Manager](#) in later scenes**, to avoid conflicts and duplicated behavior.

Testing interactions in later scenes

If you need a manager just to test a single scene, you can create a **Temporary Manager**. In the Editor, it **auto-destroys** as soon as you leave that scene. In a build, if a **Temporary Manager** is present, it **destroys itself** in `Awake`.

To create one: `Tools > Fast Studios > PressE PRO > Create Temporary Manager`

Asset Structure

This asset has two primary scripts that handle about **75%** of the interaction work: [Interaction Manager](#) and [Interactable](#). The [Interaction Manager](#) orchestrates interactions (detects the player, shows prompts, and runs the logic). The [Interactable](#) supplies the configuration for that specific interaction (mode, UI, events, conditions).

There are also other scripts, such as the [UI Prefab](#), which tells the system **which UI elements** (text, image, slider, etc.) to display and lets you customize the interface.

See the [Components](#) section for other components and additional details.

Interaction Types

- [Unity Event](#) — run any UnityEvent.
- [Hold](#) — press and hold for X seconds to trigger.
- [Grab](#) — pick up, carry (optionally physics-based), throw (with optional charge & trajectory).
- [Drag](#) — move along one axis (Position or Rotation). Great for drawers, doors, valves.
- [Inspection](#) — pick up to inspect; optionally rotate, show Text and/or Image details.

Unity Event

Unity Event is one of the simplest modes: configure a **UnityEvent** to run any action, such as: play sounds, enable/disable a GameObject, or call a method on a script. When the player interacts, the event fires **immediately**.



Hold

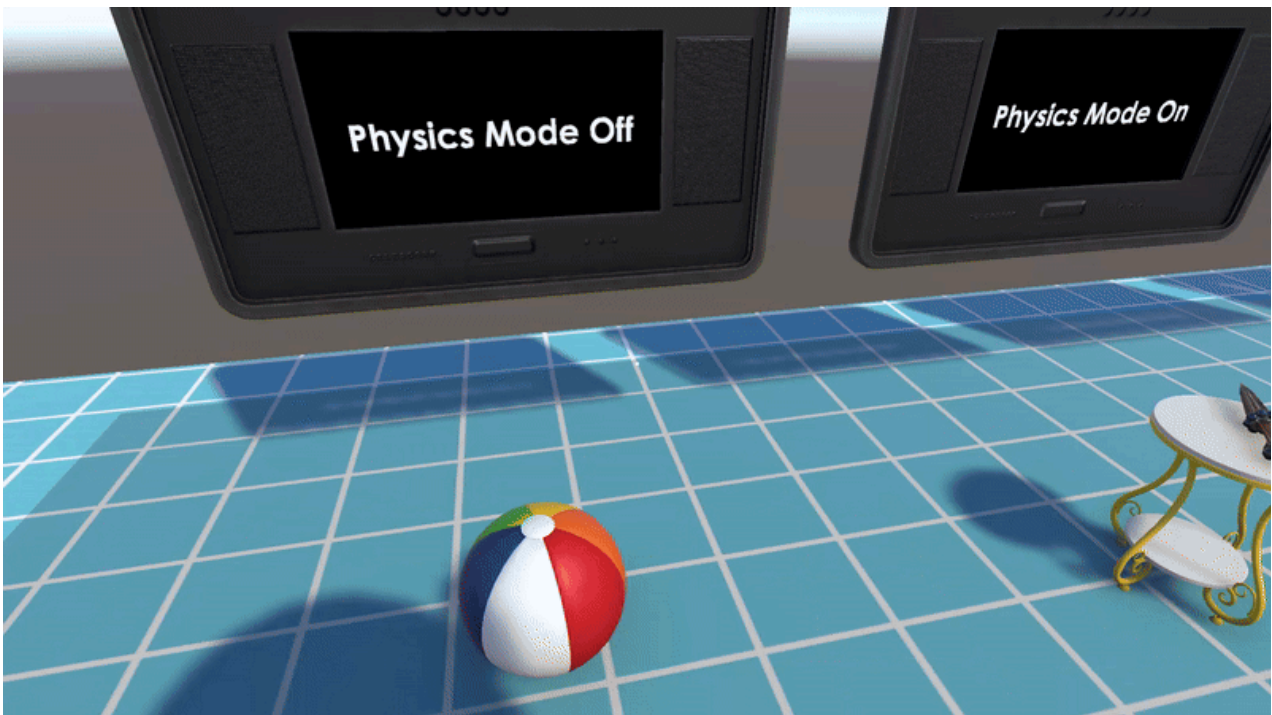
Hold triggers the [UnityEvent](#), but the player must **hold for X seconds** before it fires. For visual feedback, enable a **progress Slider** to display the **percentage of elapsed time** relative to the total duration.



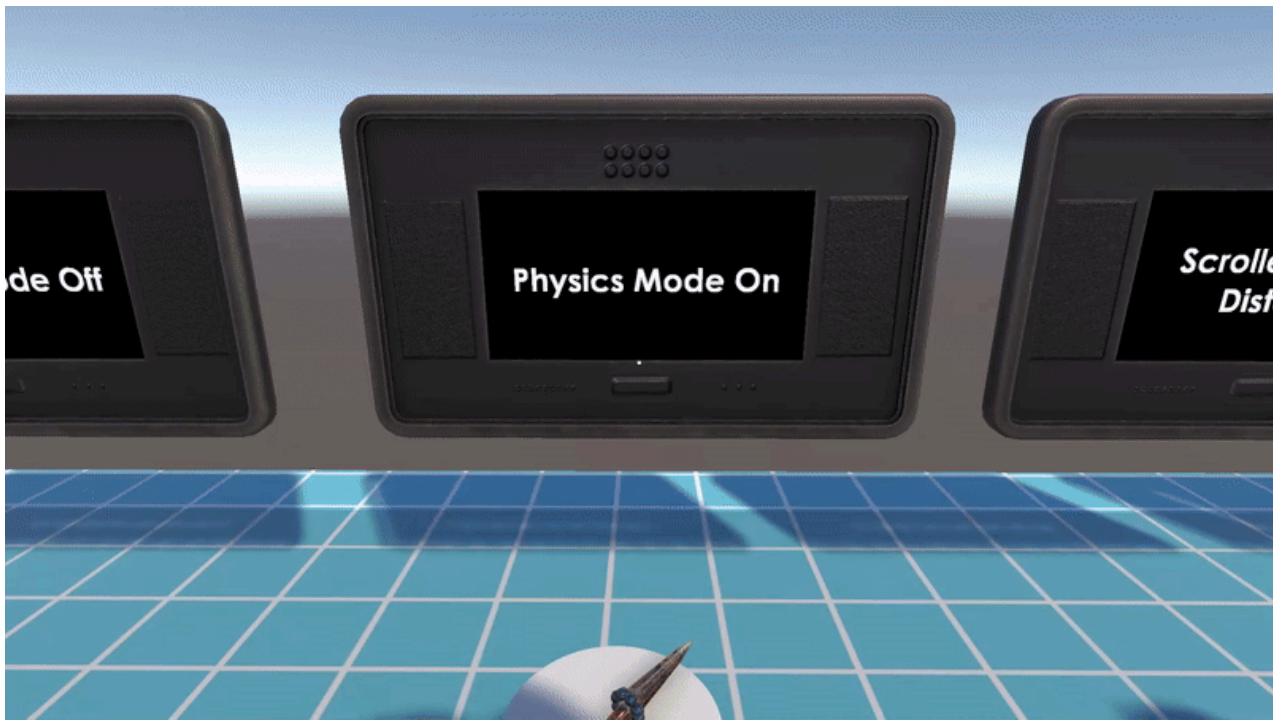
Grab

Grab lets the player **pick up** an object and **control it** while carrying it. There are **multiple grab styles** (e.g., simple carry, physics-assisted carry, and throw options with trajectory preview). **See the GIFs below** for each style.

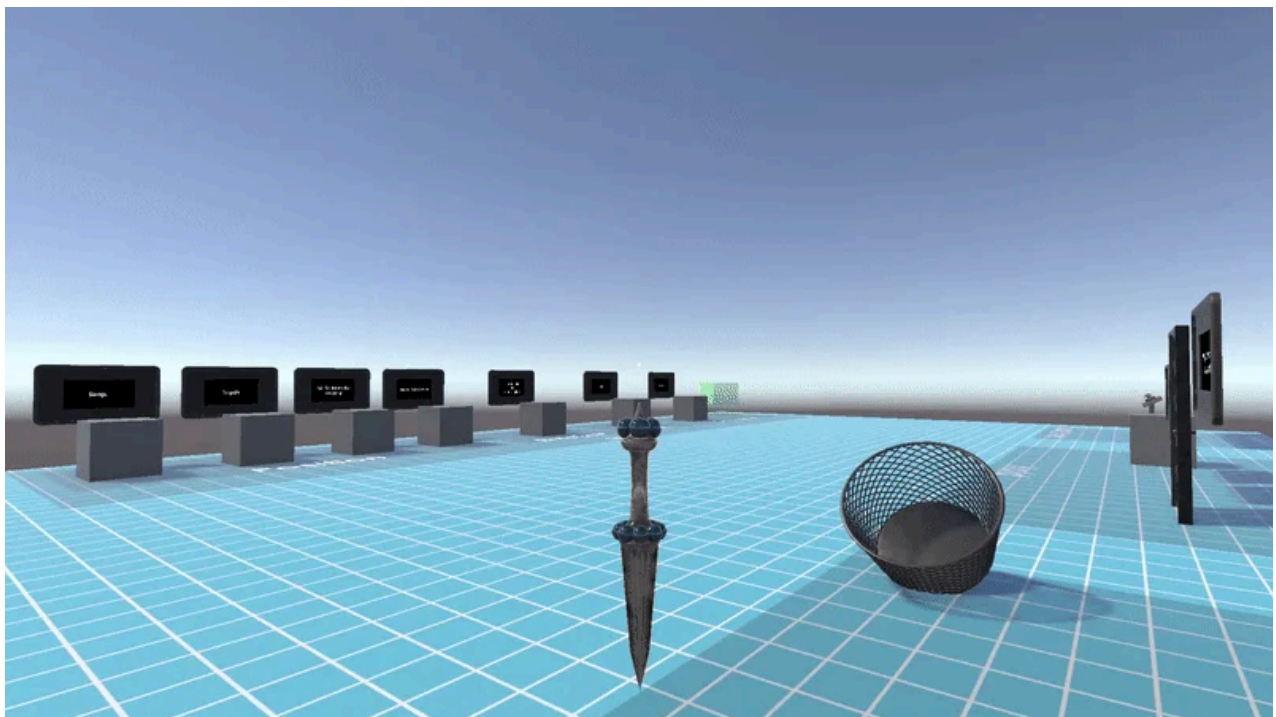
Normal Grab



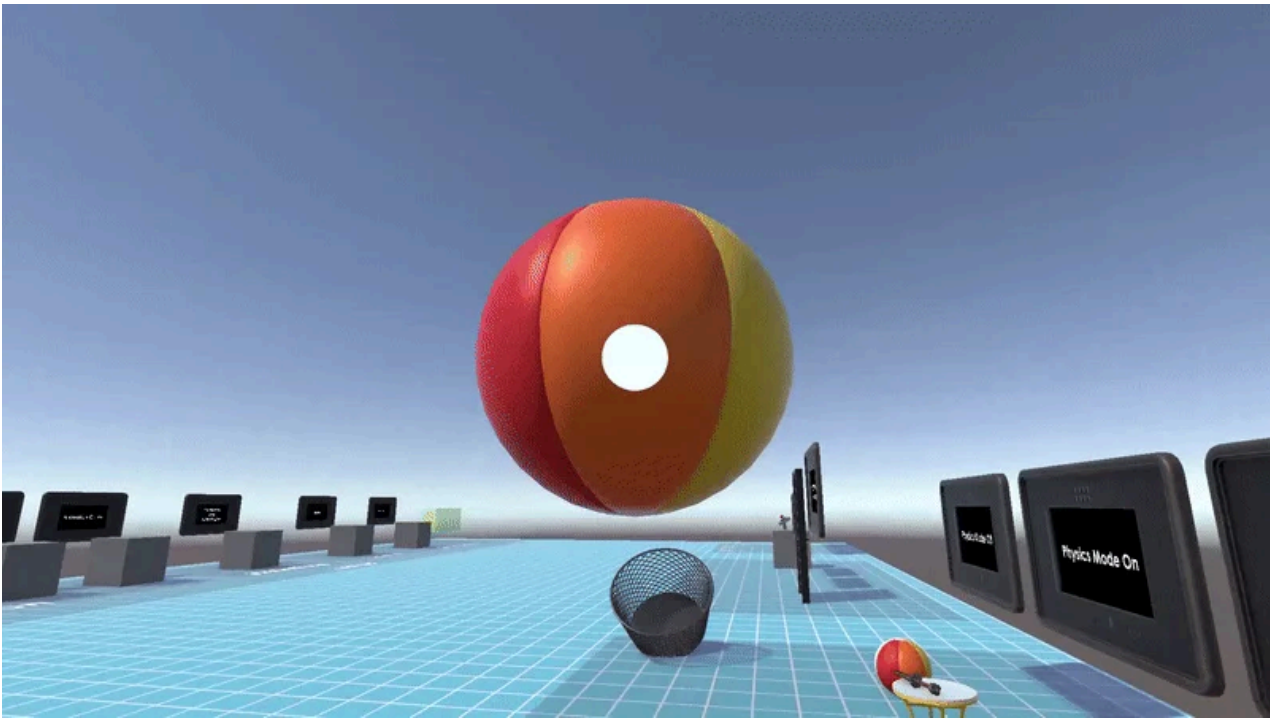
Physics Grab



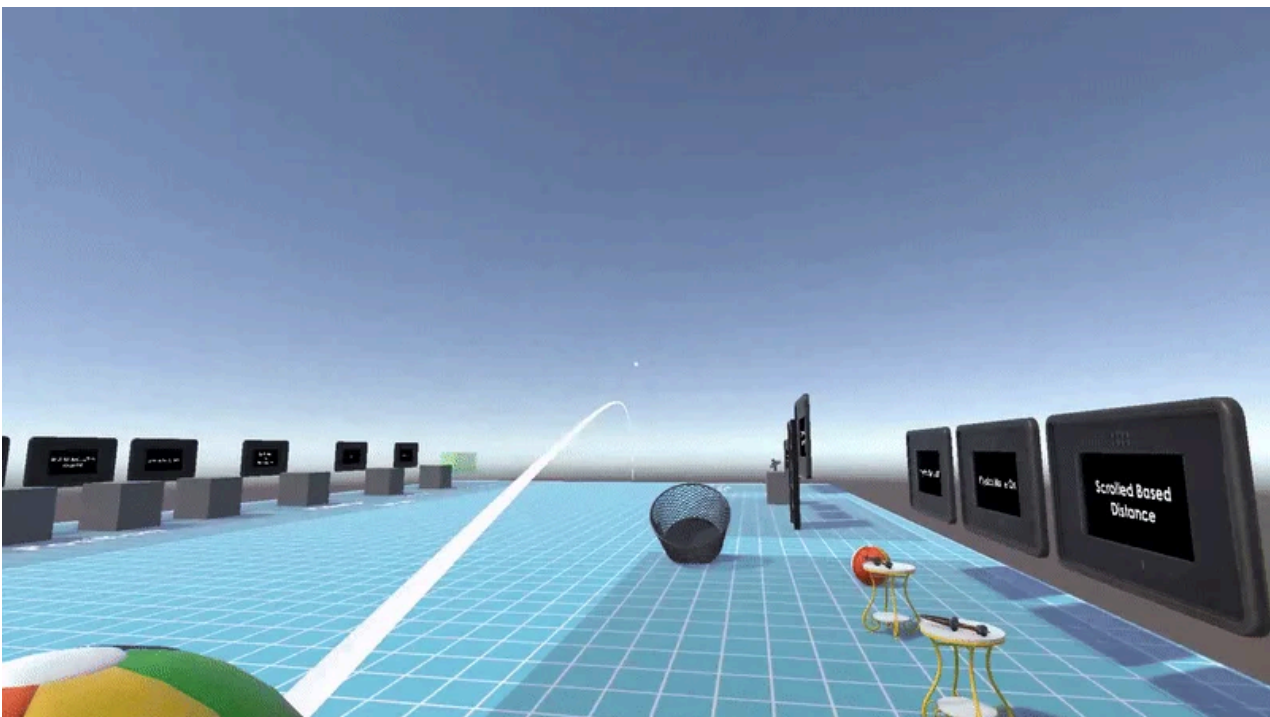
Scrolled Distance



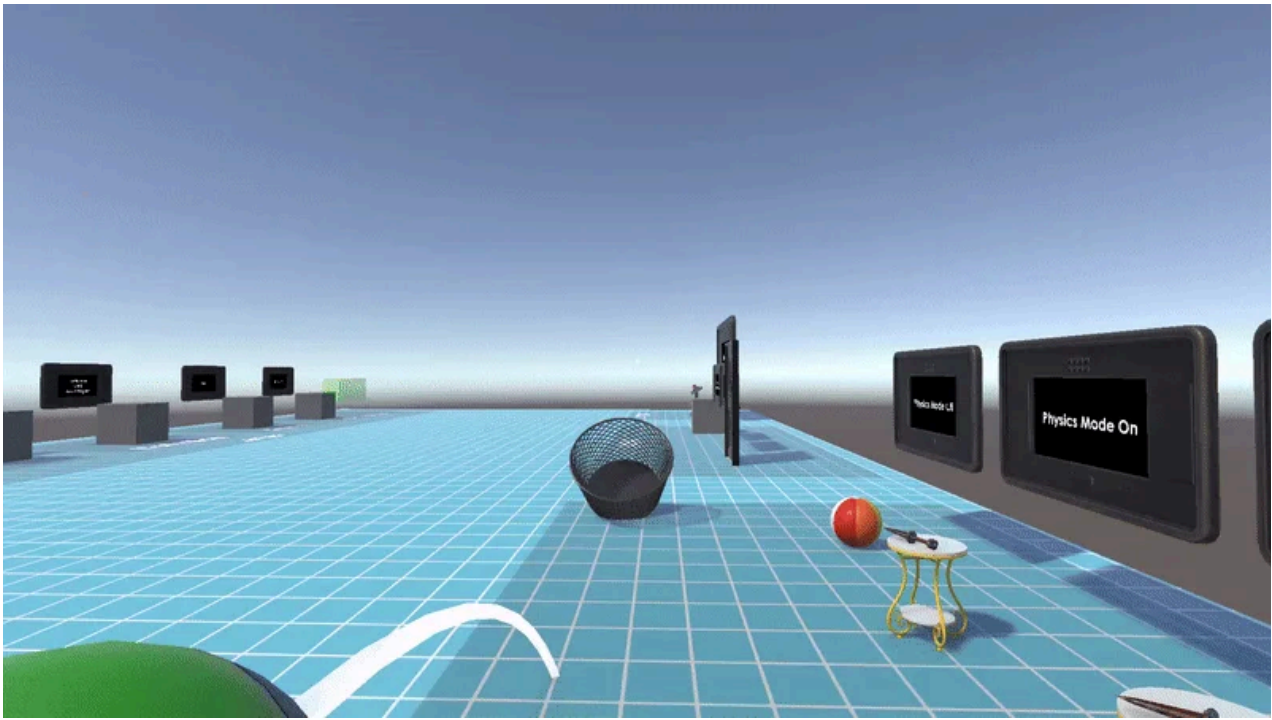
Throwable



Trajectory Viewer



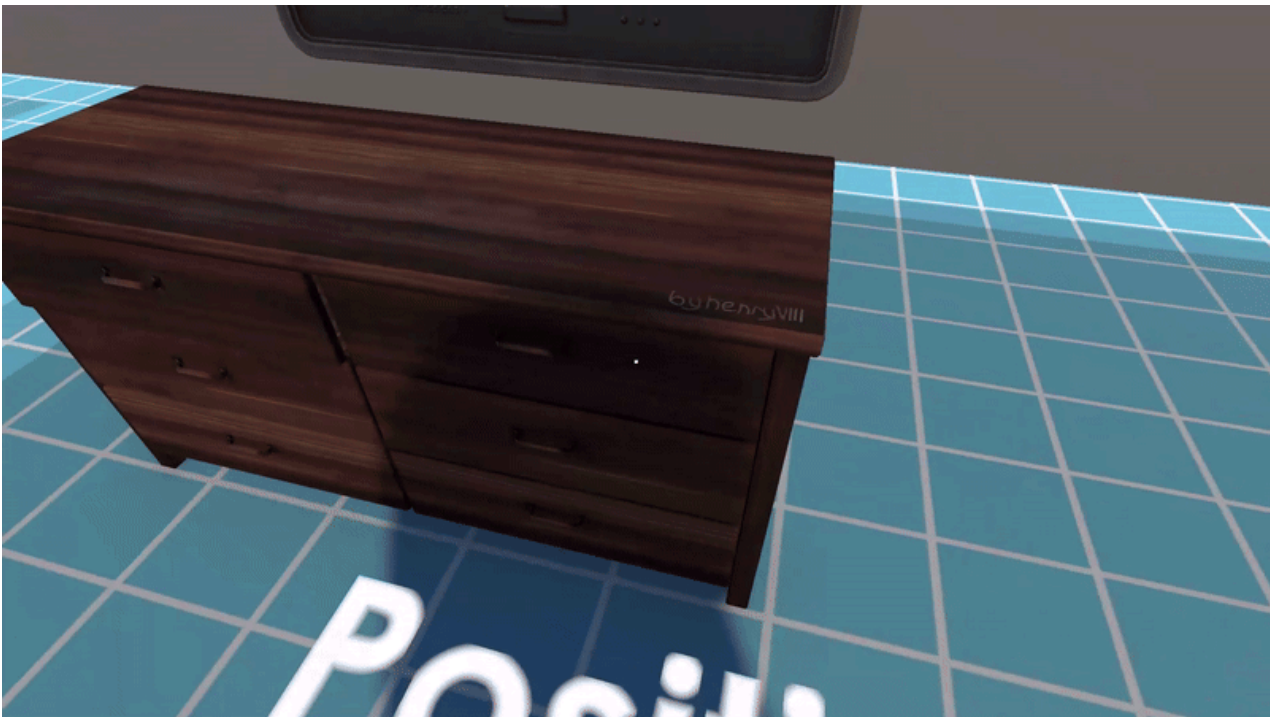
Hold Charge



Drag

Drag is a one-axis interaction with two variants: **Position** and **Rotation**. It operates on a **single axis (1D)**, making it ideal for doors and drawers (**Position**) or levers and valves (**Rotation**). Choose the variant based on whether you need **linear movement** or **rotation**.

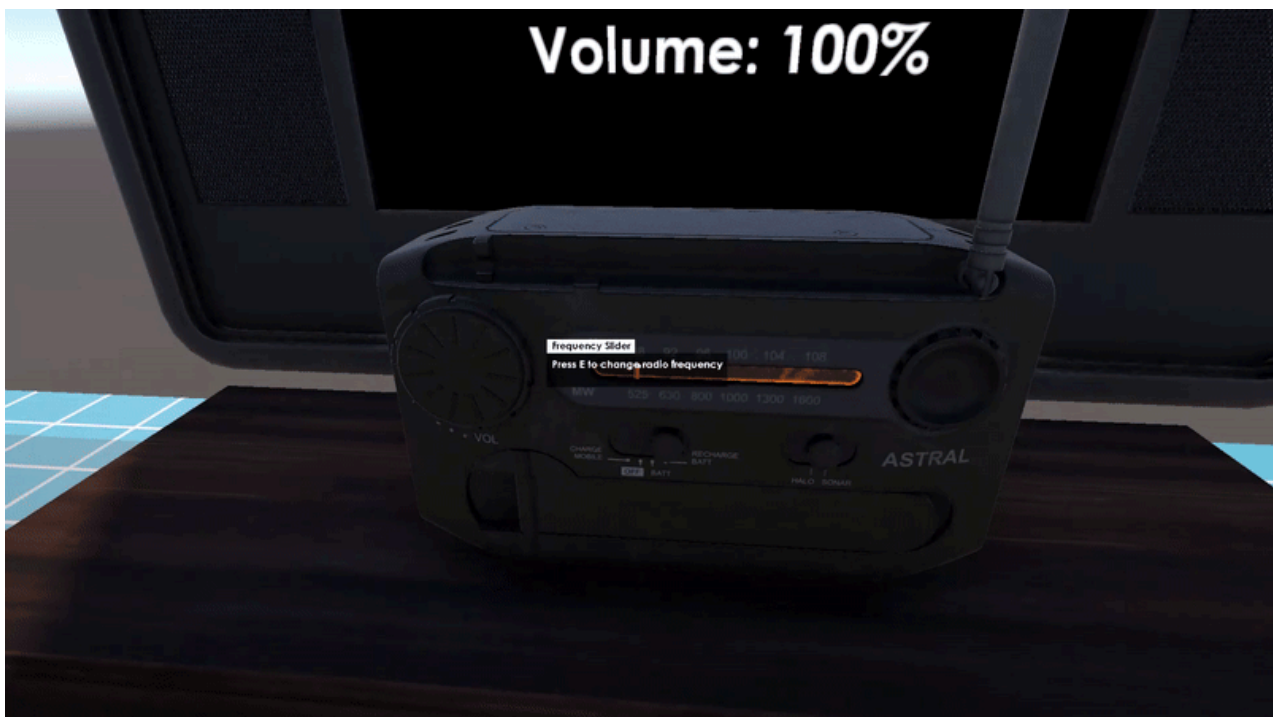
Drag Position



Drag Rotation

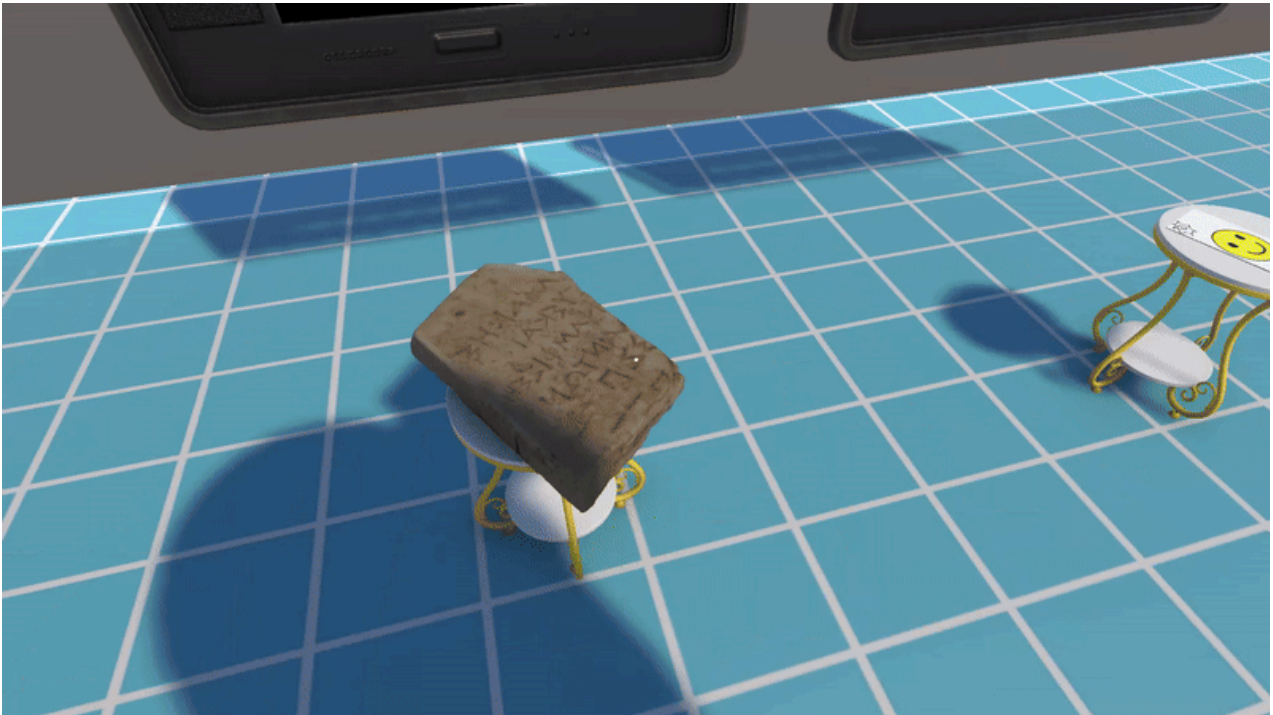


Slider Value



Inspection

Inspection is perfect for mystery/exploration games with notes, collectibles, or items the player can **pick up and examine** (rotate and bring closer). It includes **Details**, which lets you **show text and/or an image** about the inspected item. You can **toggle** these details during inspection (via a configured key) and optionally enable a **background** to improve readability.



Can Rotate



If **Image** and **Text** are toggled by the **same key**, you can set the order to show **one first** and then **the other**. If **neither Text First** nor **Image First** is enabled, the system will display **both at the same time**.

Text First

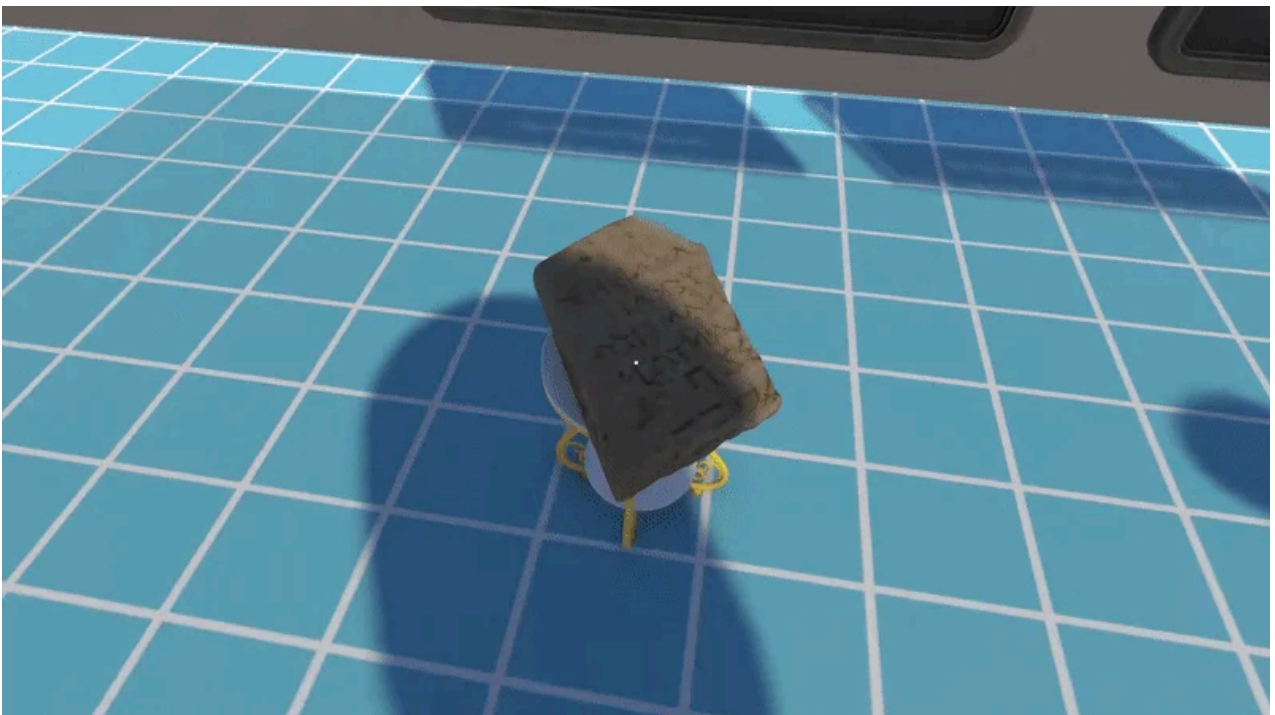
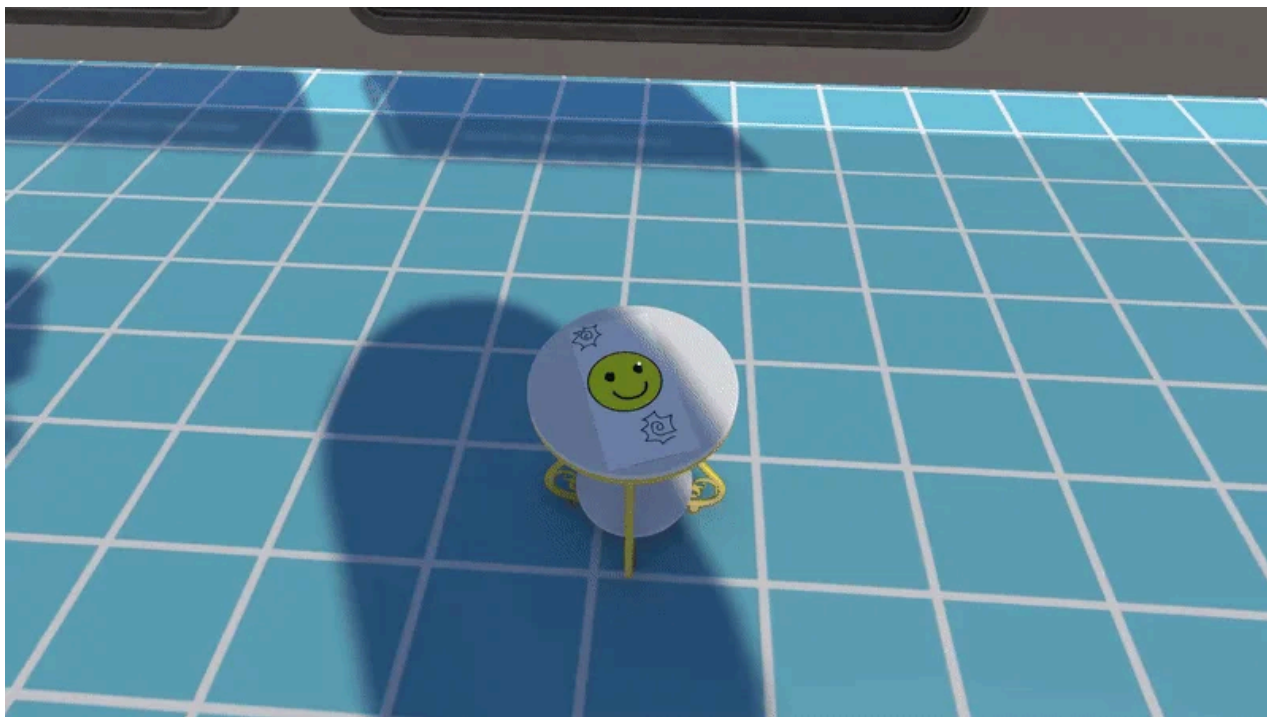


Image First

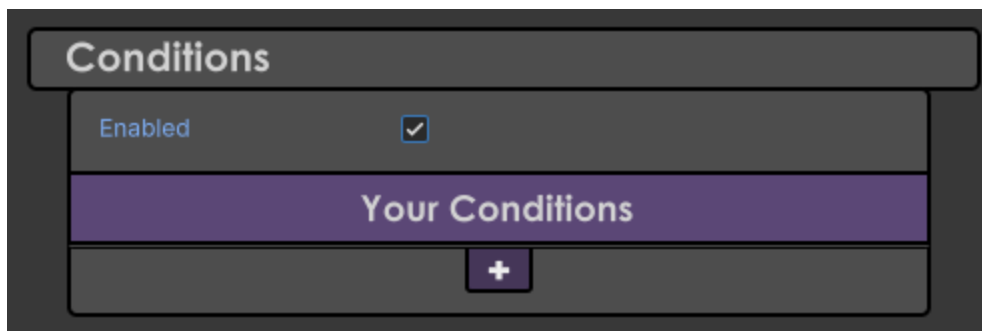


Conditions

To make setup **faster and more flexible**, the system can **read/sync variables** (not every type, but the common essentials) from **any script/component** in your scene. Simply **drag the object**, pick the **component**, and then select the **variable**. There are **three main** condition types:

- **Constant** — compares a variable to a **constant value**.
- **Dynamic** — compares **one variable** to **another variable**.
- **Key** — checks whether the player **already has** the key.

To enable **conditions**, open the **Main Tab** at the [Interactable](#) inspector and turn on the **Conditions** foldout (it's always the **last** one). When enabled, you'll see the **"Conditions"** header and the **Add** button. Click **Add** to insert a condition, and repeat to **add multiple** items to the list.



The **left column** controls the **order** of the condition list. **Between conditions** there's an **AND/OR** selector that defines how they combine. To **reorder**, use the controls at the **top left** (use the arrows). To **remove** a condition, click the – button at the **bottom left**.

The screenshot shows the 'Your Conditions' panel with the following configuration:

- Condition 1:**
 - Read From: PressE Manager
 - Component: InteractionManager
 - Variable: playerObject
 - Operator: Not Null
 - Dynamic: ☐
- Operator:** And
- Condition 2:**
 - Read From: Radio
 - Component: RadioManager
 - Variable: TurnOn
 - Operator: !=
 - Reference: Radio
 - Component: AudioSource
 - Variable: mute
 - Dynamic: ☒
- Operator:** Or
- Condition 3:**
 - Key Check Method: Key Name
 - Key Name: Key 1
- Operator:** And
- Condition 4:**
 - Key Check Method: Specific Key
 - Specific Key: Key (Key)

Normal Condition

For a **normal condition** (Constant/Dynamic), the **left column** shows **3 fields**:

- **Read From:** the object that holds the script/component with the target variable.
- **Component:** select the component/script that contains the variable.
- **Variable:** choose the variable to read/observe.

In the **middle**, next to **Variable**, pick the **operator** (available options depend on the variable type). Supported operators:

- **==**, **!=** — **string**, **bool**, **object**, **numbers** (*float*, *int*, *double*, etc.), **Quaternion**, **Vector2**, **Vector3**.

- `>`, `>=`, `<`, `<=` — **numbers**, **Vector2**, **Vector3** (compares using the vector's *magnitude*).
- `X=`, `Y=` — **Vector2** and **Vector3**.
- `Z=` — **Vector3** only.
- `Contains`, `Starts With`, `Ends With` — **string** only.
- `Is True`, `Is False` — **bool** only.
- `Is Null`, `Not Null` — **object** only.

The **right column** defines **what** the left variable is compared against.

- If **Dynamic** mode is **enabled**, you can **select another variable** on the right side.
- If it's **disabled**, a **constant value** field of the appropriate type is shown.
- **Note:** depending on the operator (e.g., `Is Null`, `Is True`), the right side **may be hidden** because it isn't required.

Key Condition

To set up a **key condition**, click the **key icon button** in the **bottom-right corner** of the condition item. The condition switches to **Key** mode and shows only:

- **Key Check Method: Key Name | Specific Key**

Choose **Key Name** to check whether the **provided name** matches a **key already obtained** by the player. Choose **Specific Key** to check whether a **specific** `Key` **component** (the actual script/object) matches **an obtained key**.

UI

The UI system works by **instantiating a prefab** that has the `UIPrefab` script **at the root**. The [Manager](#) includes its own **Canvas** and spawns **different prefabs** depending on the situation (see the [Interaction Manager](#) page for the cases). If an [Interactable](#) is **overriding** the [UI prefab](#), the system uses the overridden prefab.

Connect your UI

Sometimes you'll want to show something in the UI that isn't covered out of the box. You can **manually change values** by referencing the `UIPrefab`.

```
// Outside Class
using FastStudios;

// Inside Class

public UIPrefab UiPrefab;

public void ChangeBackgroundColor(Color newColor)
{
    UiPrefab.Background.color = newColor;
}

// Or if you dont want to create a field, you can get from your object
public void ChangeInteractionText(GameObject uiPrefab, string newText)
{
    uiPrefab.GetComponent<UIPrefab>().InteractionTmpText.text = newText;

    // For more safety you can use TryGetComponent

    if (uiPrefab.TryGetComponent<UIPrefab>(out var UiPrefab))
    {
        UiPrefab.InteractionTmpText.text = newText;
    }
    else
    {
        Debug.Log($"UI Prefab missing on: {uiPrefab}");
    }
}
```

Useful Scripts

You can add scripts **inside the UI prefab** so it behaves correctly for any interaction type, for example:

- **UI Interaction Conditions** — controls whether a **UI object** is **active or not** depending on the **interaction type** that spawned the UI Prefab. Example: you have a fixed text in the prefab and you only want it **during Drag**. Add **UI Interaction Conditions** script to the Text and configure what happens **when Drag starts/ends**; the script will **enable/disable** the object accordingly.

Speed Up Workflow

To speed up your **workflow**, use a few built-in features from this system and from Unity.

Universals

Universals are default values you define once to **drastically reduce setup time**.

Changing an universal

To change an universal value, open: **Tools > Fast Studios > Press E PRO > Universals Window**

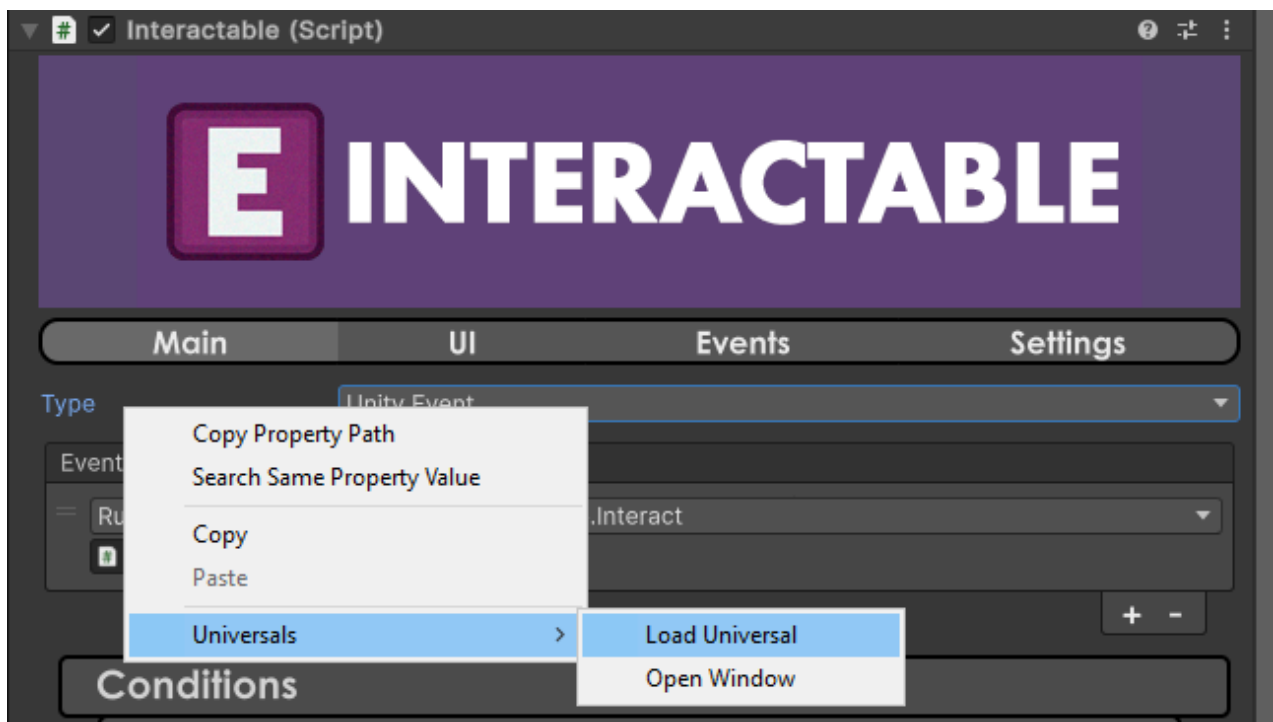
In that window, the **left side** lets you pick the script you want to configure (only scripts that support universals are listed).

Below the selector there's a **Reset** button that restores all variables to factory defaults.

On the **right side** you'll find the foldouts with the available variables; enter the values you want. If you enable **Load On Creation**, any newly added component of that script will automatically load these universal values.

Loading universal value into a variable

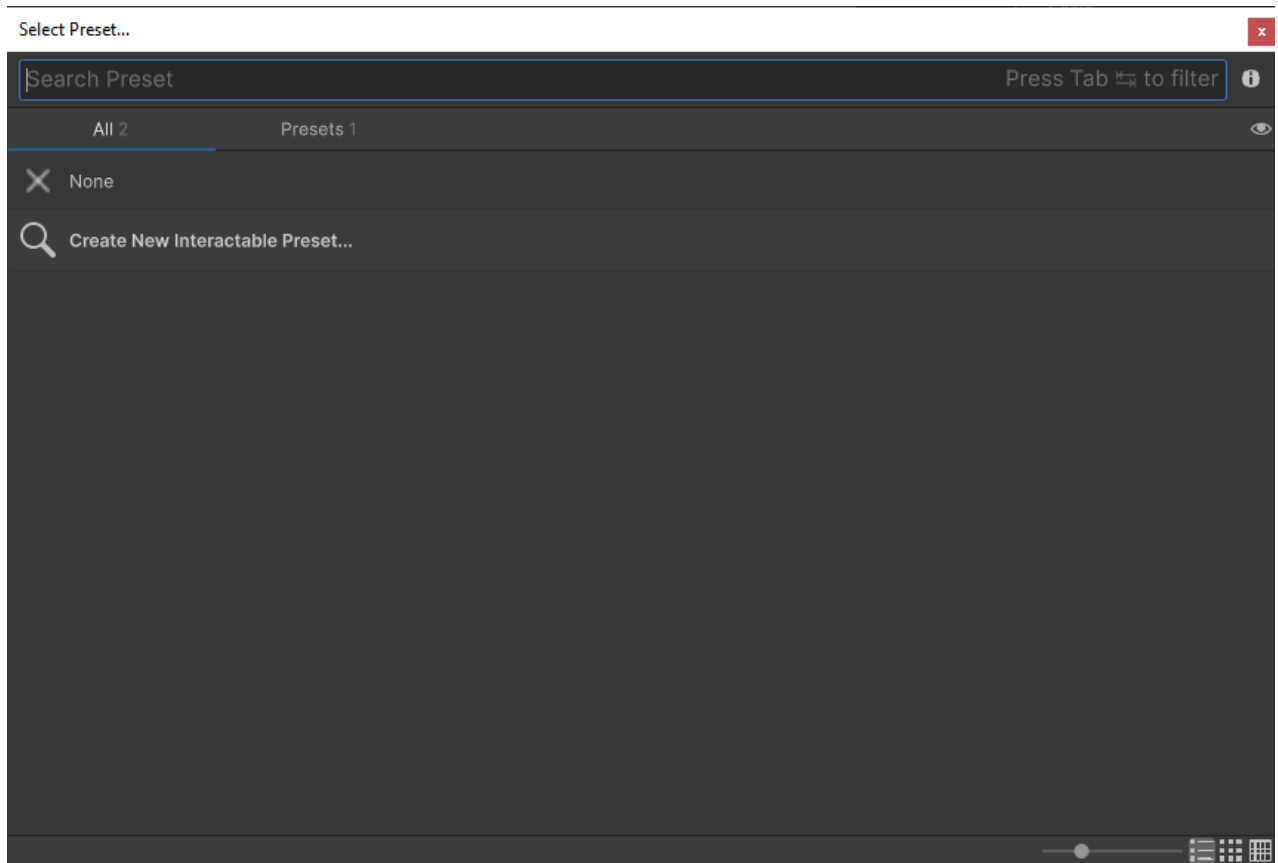
To **load a variable** into a compatible Inspector field, right-click the field and choose **Universals > Load Universal**. The field becomes read-only and uses the global value. To revert to the previous local value, right-click again and select **Universals > Unbind Universal**. This lets you switch between the universal and the component's own value whenever you need.



Presets

We also recommend Unity's built-in **Presets**. Click the **Preset icon** on the component header (to the right of the script name) to open the window, where you can apply an existing preset or create your own. It's an efficient way to save frequently used configurations and reuse them quickly.





Components

These are the included components in **Press E Pro**:

- [Interaction Manager](#)
- [Interactable](#)
- [Key](#)
- [Grab Deposit](#)
- [Lerps](#)
- [UI Prefab](#)

Interaction Manager

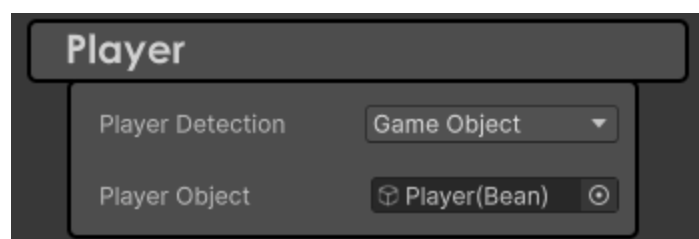
This script is the **core controller** for interactions. Without it, interactions in the scene **won't** work. We recommend placing it in the **first scene** of your Build because it's a **singleton** with **Dont Destroy On Load**: there must be **only one**, and it stays loaded across scenes.



Main Tab

- **Key To Interact:** the key used to interact.
- **Interaction Distance:** the maximum distance at which the player can interact.

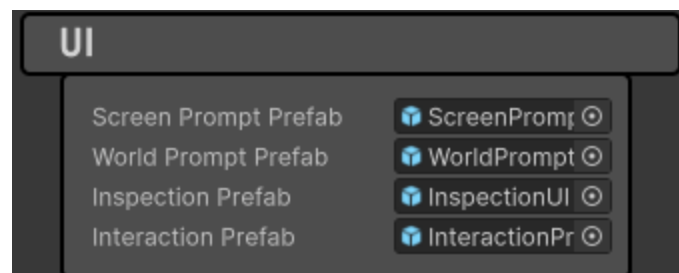
Player Foldout



This foldout defines **how the player object is found**. Available modes:

- **Game Object:** direct reference to the player object. If the player object is **not** DDOL, you must reassign it after changing scenes.
- **Tag:** finds an object with the chosen tag to be the player.
- **Layer:** finds an object with the chosen layer to be the player.
- **Object Name:** finds an object by **name** to be the player.
- **MonoBehaviour:** finds an object that has the chosen **MonoBehaviour** script.
- **Reference:** in each scene that has a player, add a **Reference** script (it doesn't need to be on the player). Assign the player in that Reference; on **Start**, it will automatically set the current player object.

UI Foldout

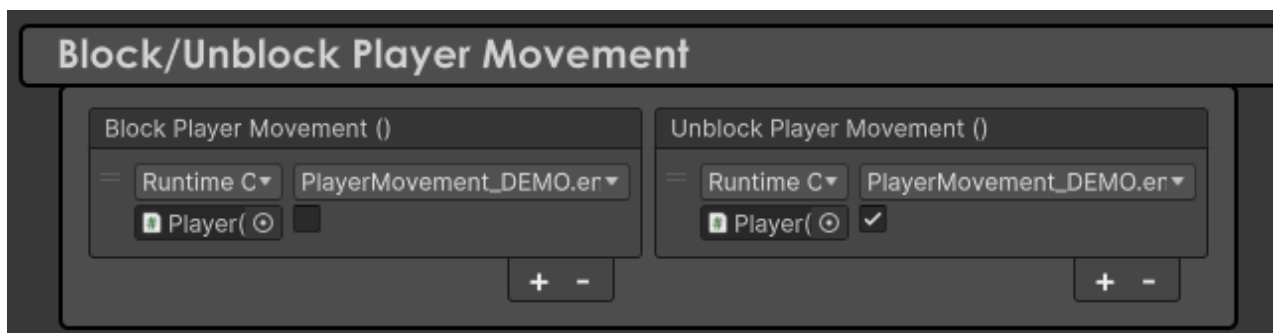


Defines the **default UI prefabs**. Whenever a UI type is displayed, the Manager instantiates the matching prefab.

- **Screen Prompt Prefab** → **Screen Prompt:** on-screen notice (e.g., *Press E to interact*).
- **World Prompt Prefab** → **World Prompt:** world-anchored notice positioned relative to an object.
- **Inspection Prefab** → **Inspection:** inspection UI; can show **text** and/or an **image**.
- **Interaction Prefab** → **Interaction:** UI shown **during** interactions (e.g., while carrying/dragging); if enabled, it stays visible for the duration.

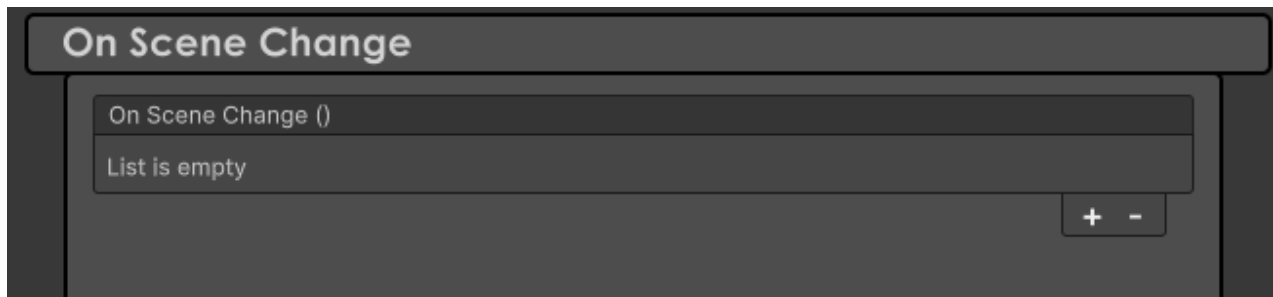
Events Tab

Block/Unblock Player Movement



Define **when** and **how** to **block/unblock** player movement (e.g., when starting an *Inspection*). You can drag **any function or component** here, it's a standard **UnityEvent**.

On Scene Change



Optional event for running **specific functions** when a **scene change** occurs.

Interactive

This script defines **which interaction type** a specific object uses. Add it to **every object** that should be interactive. For behavior details of each mode, see **Interaction Types**.

The selected interaction type determines **which fields are shown**, only what's needed for that mode. In this documentation, the specific settings are split into four subpages:

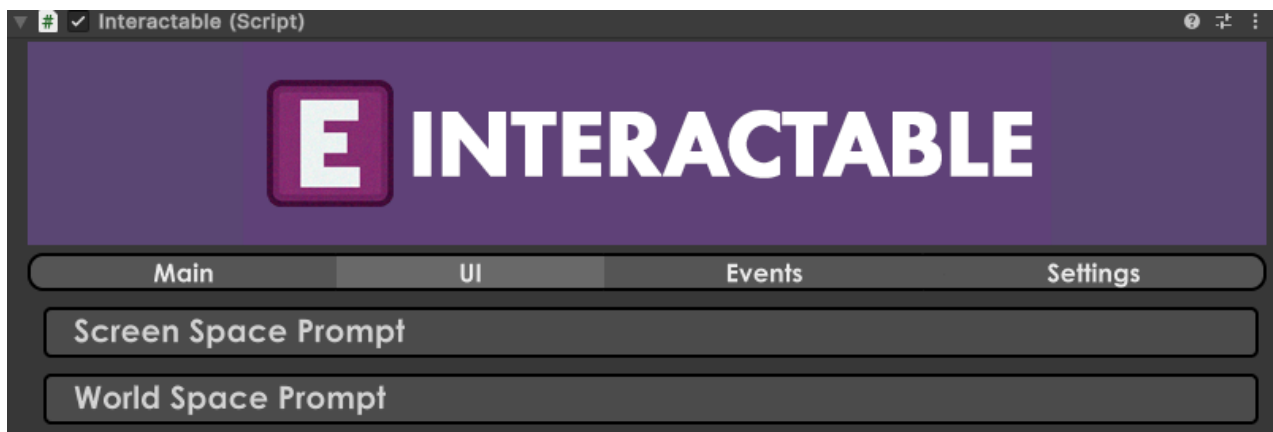
- [Unity Event](#)
- [Hold](#)
- [Grab](#)
- [Drag](#)
- [Inspection](#)

Main Tab

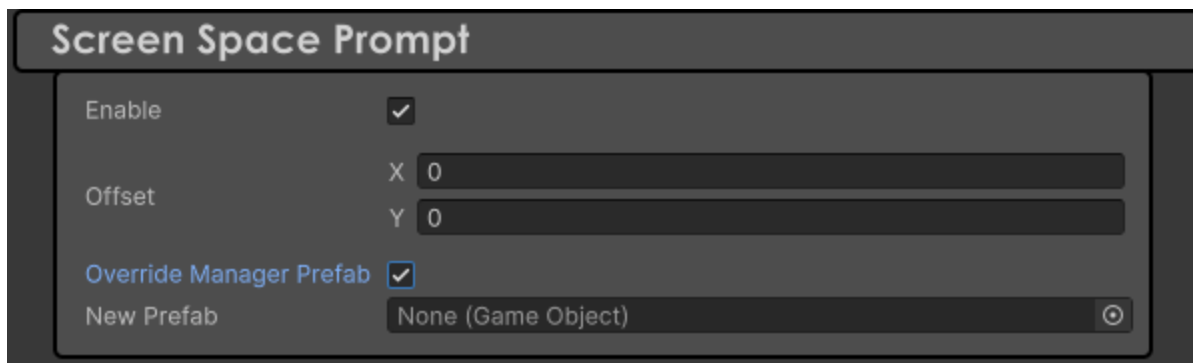
All interaction types expose the **Conditions** foldout. Enable it to reveal the list and **add** as many conditions as you need. See [Conditions](#) for details.

UI Tab

Everything related to **UI** lives here. For any interaction type, you can enable **Screen Space Prompt** and **World Space Prompt**. See the [UI](#) section for more details.

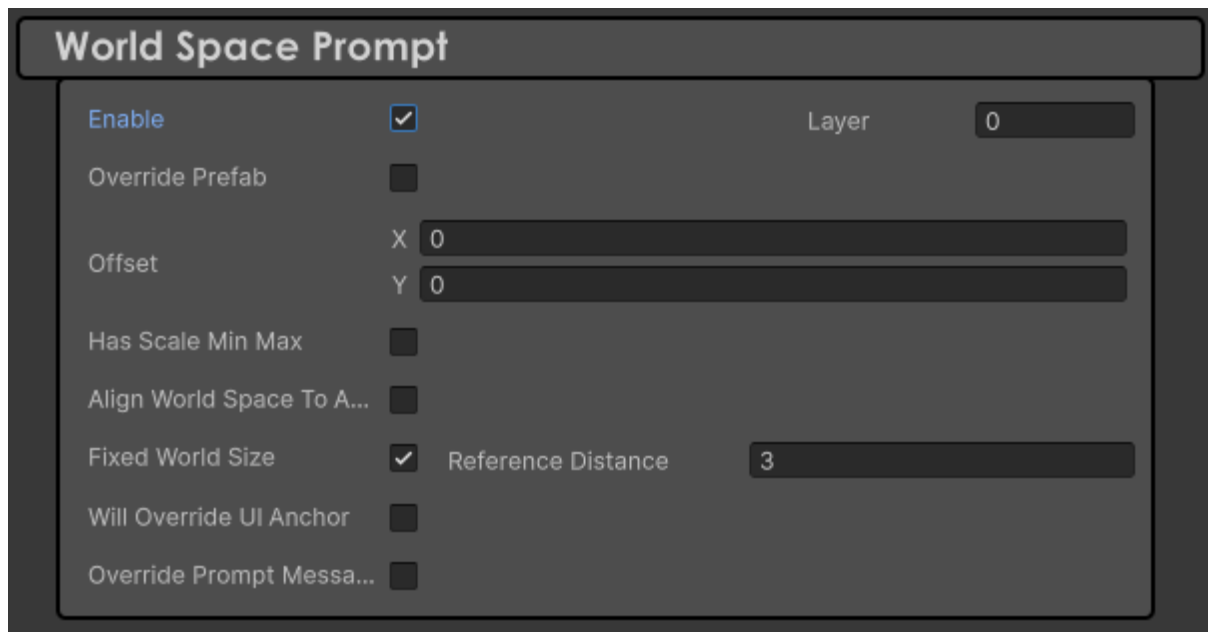


Screen Space Prompt



- **Offset:** apply a local offset for this interaction.
- **Override Manager Prefab:** have this interaction use a **specific** prefab. (The prefab must include [UIPrefab](#).)

World Space Prompt



Open the foldout and check **enabled**. Key fields:

- **Layer:** useful when multiple UIs exist via the sensor and you need **priority** (higher layer = higher priority).
- **Override Prefab:** use a **specific prefab** for this interaction. (*Prefab must contain [UIPrefab](#).*)
- **Offset:** local offset for this interaction.
- **Has Scale Min Max:** since the position adapts to the **object**, scale can change; set **min** and **max** bounds.
- **Fixed World Size:** when **on**, the UI appears with a **constant size** regardless of distance. When **off**, the size visibly varies with distance.
- **Will Override UI Anchor:** UI uses the object's **default anchor**; if the Interactable has a **new anchor**, it will use that. If you need a **UI-specific anchor**, assign it here. You can type the values, click the button to **create** a new anchor in place, or drag a **Transform**.
- **Override Prompt Message:** replaces the message shown in **Interaction Text** of the **UIPrefab**.
- **Condition Declined** (*when **Conditions** is enabled*): message displayed when the condition is **not** met.

Events Tab

- **On Raycast Enter/Exit:** great for outline/highlight effects.
- **On Interact Event:** fired when the player presses the interaction key. (*Different from **Unity Event** mode, which fires immediately.*)
- **Condition** (when **Conditions** is enabled):
 - **Decline:** called when the player tries to interact and **fails**.
 - **Accept:** called when the player tries to interact and **succeeds**.
 - **On Condition Attended:** called when the requirement becomes met (e.g., the player just collected the last required key).

Settings Tab



Override Key

If this object needs a **different keybind** to interact, set the override here.

Max Interactions

Define a **hard limit** of interactions (e.g., permanently locking a door after X uses).

Sensor

Use when an interaction may be triggered from **farther away** and/or without looking directly at the object.

- **Type: Radius | Distance:** choose the **check method** (math radius check / inside-radius vs. **Unity's Vector3.Distance**).
- **Radius/Distance:** sets the **radius/distance** to interact.
- **Only Show UI:** when **on**, the player **can't interact** just by being inside the radius/distance (outside the default "look-at" rule), but the UI **will appear** once inside the allowed range defined by the Manager.

Auto interact

When enabled, looking at the object triggers the interaction **automatically**. If a **Sensor** exists, entering the sensor can also trigger it.

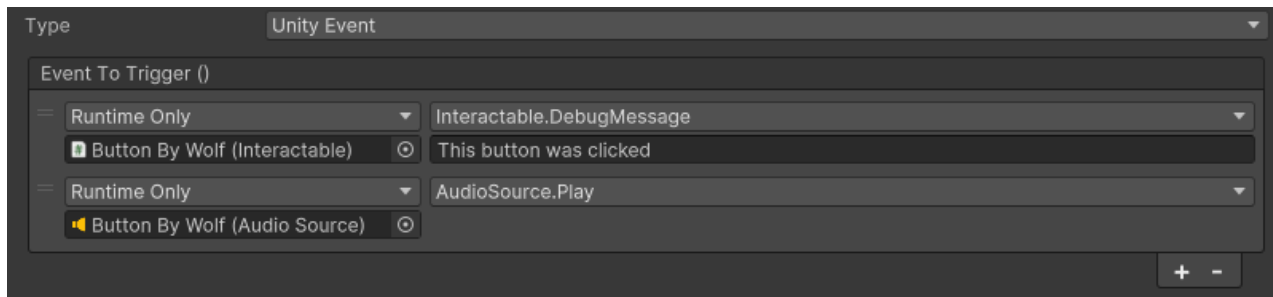
New Anchor

To set a **new anchor** (e.g., for a door), enable the feature. You can **type** values, **click the button** to create an anchor in place, or **drag** a Transform. See **Setting new anchors** for more.

Gizmos

Depending on the mode/configuration, helpful **gizmos** appear to assist setup. You can **enable/disable** them here and choose to show gizmos **only when selected**, etc.

Unity Event

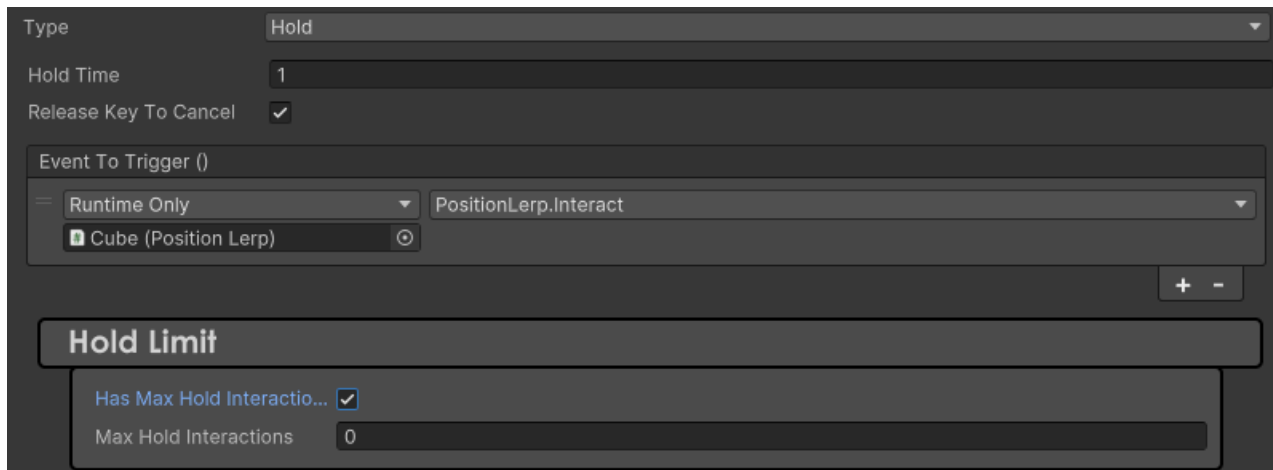


Main Tab

Contains just the unity event.

Hold

Main Tab



- **Hold Time** — required **time** the player must hold before the event fires.
- **Release Key To Cancel** — when **enabled**, releasing the key **cancels** progress; when **disabled**, progress **continues** even if the key is released.
- **Event To Trigger ()** — **UnityEvent** invoked when the **Hold** completes.
- **Hold Limit** — maximum number of **successful** Holds before stopping the Hold behavior (after the limit, the event occurs without Hold, depending on your setup).

UI Tab

While Interacting

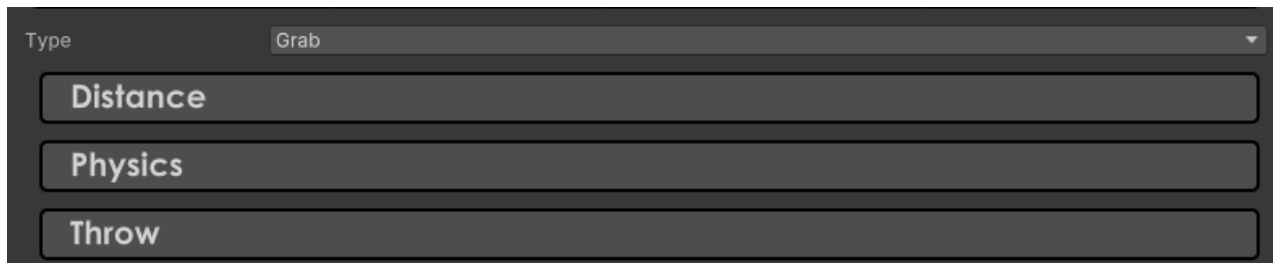
When using **Hold**, the **While Interacting** foldout appears under **UI Tab**:

- **Has Slider** — instructs the [UI Prefab](#) for While Interacting to **show a progress Slider**.
 - **Apply Slider Value** — when enabled, the system **automatically applies** the progress value to the **UI Prefab**'s Slider.

- **Override Prefab** — assigns a **specific prefab** for the While Interacting UI (the prefab **must** include [UIPrefab](#)).
- **Is World** — when enabled, the While Interacting UI behaves as **World UI** (world-anchored).
 - **Will Override UI Anchor** — if **Is World** is enabled and you need to change which **anchor** the [UIPrefab](#) follows, set it here.

Grab

Main Tab



Distance

Controls where the object sits relative to the player.

- **Distance:** sets the **distance** between player and object.
- **Offset:** applies a positional **offset** (left/right, forward/back, up/down).
- **Scrolled Based Distance:** when **enabled**, the **scroll** adjusts the distance.
- **Scroll Multiplier:** how strongly the scroll **affects** the distance.
- **Distance Min Max:** clamps the distance with **min/max** so scrolling can't push it too far.

Physics

You can enable physics-based grab in this foldout, and it is importante to have a **Rigidbody**, because physics influences.

- **Player Helper Force:** helper force to keep the object **centered/stable**.
- **Linear Damping:** higher values mean **less positional wobble**.
- **Angular Damping:** higher values mean **less rotational wobble**.

Throw

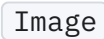
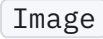
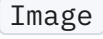
Makes the object **throwable** when Throw is enabled.

- **Force**: strength applied to **throw** the object.
- **Throw Towards Aim**: **on** = throw **toward aim**; **off** = throw **straight ahead**.
 - **Distance (When Use Raycast Aim is false)**: distance used when not aiming with raycast.
- **Use Raycast Aim**: uses a **raycast** to aim and applies force **toward the hit**.
- **Hold Charge**: holding the button **charges** throw force.
 - **Force Clamp**: **min/max** limits for force.
 - **Time to Reach Max Force**: time to reach **max strength**.
 - **Use Curve**: apply a **curve** to shape the charge growth.
- **Can See Trajectory**: shows a **trajectory preview**.
 - **Points**: number of **points** on the line.
 - **Precision**: trajectory **accuracy** (100% = most detailed).
 - **Material**: line **material** (use your custom if desired).

UI Tab

While Interacting

When using Grab, the **While Interacting** foldout appears under **UI Tab**:

- **Override Image Sprite**: change the **sprite** on the **UIPrefab's** .
- **Override Image Color**: change the **color** on the .
- **Override Image Size**: change the **size** on the .
- **Override Interaction Prefab**: replace the **interaction prefab** to be instantiated.

Events Tab

- **On End Event**: invoked **when the player releases** the object.

Drag

Drag has two types: **Position** and **Rotation**. It's always **1D** (a **single axis** per interaction). Because model export/orientation can differ, **enable gizmos** to verify the chosen axis is correct.

Main Tab

- **Type**: selects the drag type (**Position** or **Rotation**).
- **Axis**: chooses **which axis** is affected. Since drag is 1D, pick from **Forward/Back**, **Left/Right**, or **Up/Down**. Use gizmos to confirm orientation (models don't always map "Left/Right" as expected).

Distance

Sets the **maximum interaction distance**. Beyond this range, you **no longer control** the object via drag.

Steps

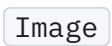
Creates **snapped positions/angles** along the 0–1 clamp.

Example: with **3 steps** on a **Position** drag, the object snaps to **0**, **0.5f**, and **1**.

UI Tab

While Interacting

When using Drag, the **While Interacting** foldout appears under **UI Tab**:

- **Offset**: applies an **offset** to the prompt's final position.
- **Override Image Sprite**: changes the **sprite** on the **UIPrefab's** .

- **Override Image Color:** changes the **color** on the `Image`.
- **Override Image Size:** changes the **size** on the `Image`.
- **Override Interaction Prefab:** replaces the **interaction prefab** to be instantiated.
- **Will Override UI Anchor:** set this if you need to **change the anchor** the **UIPrefab** follows during dr

Events Tab

- **On End Event:** invoked **when the player releases** the object.

Position

This subpage covers the settings specific to [Drag Position](#).

Main Tab

Clamp

Set the **minimum** and **maximum** travel (e.g., a drawer's range). This keeps the object within bounds and ensures consistent behavior. If needed, use **Invert Direction** so the movement starts **forward** and goes **backward**, or the other way around.

Physics

Control the physical feel while dragging:

- **Spring**: stiffness of the "spring" pulling the object toward the rail target. Higher values feel **more responsive** but can **oscillate** if **Damping** is low. Pair with **Damping** to stabilize and with **Rigidbody Damping** to smooth the rigidbody during drag.
- **Damping**: stabilizes motion; higher values mean **less positional wobble**.
- **Rigidbody Damping**: damping applied directly on the **Rigidbody** (helps prevent overshoot after release).
- **Bounciness**: how much the object **bounces** when it hits the limits at speed. (*field name preserved*)
- **Locked**: when enabled, the object **does not move** until the boolean is turned off.

Gizmos

Tune Scene View visualization:

- **See Clamp Gizmos:** shows the **clamp line** (min–max).
- **Rail Color:** color of the **rail/clamp line**.
- **Limit Color:** color of the **limit spheres**.
- **Current Pos Color:** color of the **current position** sphere.
- **Sphere Radius:** size of the **spheres**.
- **Max Distance Gizmos:** displays the **max interaction distance**.
- **Max Distance color:** color of the **max distance** indicator.

Slider

Exposes a **0–1** value representing **progress** along the clamp: **0** at min and **1** at max.
Use **Value MinMax** to set the Slider's **minimum and maximum** report values.

Rotation

This subpage covers the settings specific to [Drag Rotation](#).

Main Tab

- **Direction** — sets the **rotation direction** (clockwise/counter-clockwise). *Note: may be ignored if no clamp is used.*
- **Offset** — applies a rotation **offset** (in **degrees**).

Clamp

- **Clamp Angles** — sets the **min** and **max** rotation. Values above **360** are allowed (e.g., **720** = two full turns).
- **Has Start Position** — ensures the object **always starts** at a specific rotation.
- **Start Type** — choose how to define the start point:
 - **Percentage** — uses **0%** at the clamp min and **100%** at the clamp max (e.g., **50%** starts in the middle).
 - **Degree** — specify the start directly in **degrees**.

Physics

- **Idle Damping** — damping applied **when the player isn't dragging**.
- **Damping** — stabilizes motion; higher values mean **less wobble** while dragging.
- **Spring** — stiffness of the spring pulling toward the arc target. Higher values feel **more responsive** but can **oscillate** if **Damping** is low. Pair **Spring + Damping** for stability and add **Rigidbody Damping** to smooth the rigidbody.
- **Rigidbody Damping** — damping applied on the **Rigidbody**.
- **On Limit Spring** — spring applied **when the clamp limit is reached**.
- **On Limit Damping** — damping applied **when the clamp limit is reached**.

- **Bounciness** — how much the object **bounces** when it hits the limit at speed.
(*field name preserved*)
- **Locked** — when enabled, the object **does not move** until the boolean is unset.

Gizmos

- **Fill Arc** — fills the rotation arc when enabled.
- **Angle Interval Color** — color of the **arc**.
- **Limit Color** — color of the **limits**.
- **Current Point Color** — color of the **sphere** indicating the **current rotation**.

Radius

Controls the **arc radius** representing where the object can rotate (enable **gizmos** on the object to see the arc).

- **Automatic Radius** — when on, the system **auto-calculates** the radius (based on colliders/size). Turn off to **set it manually**.

Slider

Controls how the **Slider** represents the current rotation within the range:

- **Max Value** — the slider's maximum value when rotation reaches the **max**.
- **Max Angle** — when **Clamp** is **off**, the **maximum angle** used to map rotation → slider value.

Inspection

Main Tab

- **Inspection Distance:** sets the distance between the **camera** and the **object** when entering inspection mode.

Can Rotate

Controls player-driven rotation of the object.

- **Sensitivity:** rotation sensitivity.
- **Input Method:** input mode (**Keyboard | Mouse**).
 - **Mouse: Left | Middle | Right.**
 - **Key:** select the **KeyCode**.
- **Show Cursor:** shows the cursor when rotation starts.
- **Keep Cursor:** keeps the cursor visible after rotation stops; when off, it hides on stop.

Details

Controls extra content shown when the player presses the details key.

- **Has Background:** enables the **background** object on the **Inspection** UI Prefab.
 - **Background Color:** background color.
 - **Background:** background sprite/image.
- **Has Text:** enables the **text** object on the Inspection UI Prefab.
 - **Text Activation Key:** key to toggle the text.
 - **Text:** text content to display.
- **Has Image:** enables the **image** object on the Inspection UI Prefab.
 - **Image Activation Key:** key to toggle the image.

- **Image Color:** image color.
- **Image:** sprite/image to display.

ⓘ if **Text** and **Image** share the **same key**, use **Text First** or **Image First** to decide the order. If **neither** is enabled, both are shown **simultaneously**.

Animation

Animation for **pick up** / **release** in Inspection.

- **Animation Duration:** duration.
- **Animation Curve:** curve controlling the animation.

Overrides

- **Quaternion:** rotation (quaternion) applied when the object is **picked up** for inspection.
- **Prefab:** prefab **instantiated** for **Details**.

Events Tab

- **On End Event:** invoked when the player **drops** the object / ends inspection.

Key

Add **keys** to be used as **conditions** on objects. For instructions on using a **Key** as a condition, see **Conditions**.

Main Tab

- **Key Name:** assigns a **name** to the key; later, the system can check if the player already obtained **a key with this name**.
- **Can Interact:** when **on**, the **Interaction Manager** detects when the player looks at the Key and **auto-picks** it.
 - When **off**, you handle pickup manually—for example, using a **Unity Event** Interactable that calls the Key script's [Interact](#) method, shows **world prompts**, and/or triggers extra events.
 - Also useful to **prevent** the player from grabbing the Key **directly** without your custom logic.

Events Tab

- **On interact:** fired when **interacting** with the Key.
- **On Use:** fired when **using** the Key.

Settings Tab

Main Settings

- **Add Key When Interact:** adds the Key to the system's **default obtained-keys list** (recommend **off** if you have your own inventory/keys system).
 - **Remove Key When Used:** removes the Key from that list **when it is used**.

- **Disable Object When Interact:** makes the Key's **GameObject** disappear (deactivate) when the player picks it up.

Destroy Key When Used

Controls **destroying the Key GameObject** upon use.

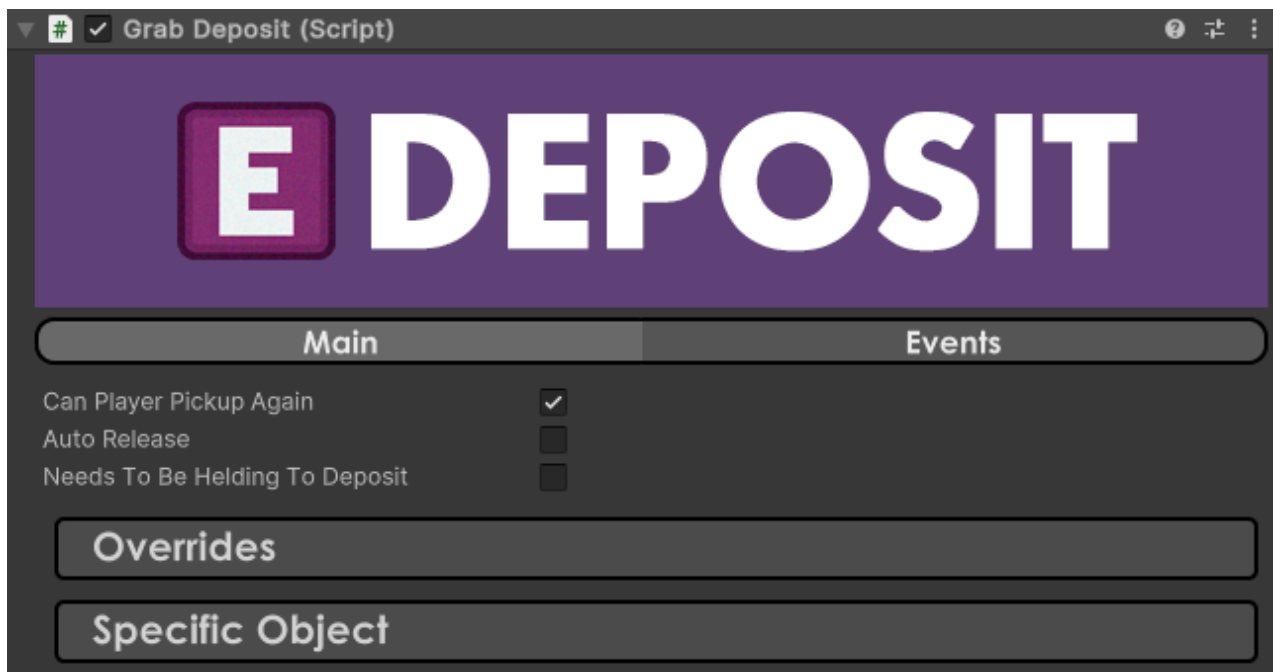
- **Specific:** restrict destruction to **specific interactions** (one or more Interactables).
- **Specific Method: Specific | Specifics:** choose **single** Interactable (**Specific**) or a **list** (**Specifics**).
- **Specific Interactable:** assign the **single Interactable** allowed to **destroy** this Key.
- **Specific List:** assign the **Interactables** allowed to **destroy** this Key.

Grab Deposit

Use this when an object must be **carried to a location** and **fixed** in place.

Requirement: the deposit object needs a **Box Collider** with **Is Trigger** enabled; this area is used to detect when a **Grab** enters or exits.

Main Tab



- **Can Player Pickup Again** — when **on**, the player can **pick up again** from inside the deposit.
- **Auto Release** — when **on**, as soon as the object **enters the collider**, it is **auto-released** and clamped. When **off**, the player must **manually release** it to deposit.
- **Needs To Be Holding To Deposit** — when **on**, the player must **be holding** the object and **cancel the grab** inside the area to deposit. When **off**, the object **can be clamped** even if it isn't in the player's hand.

Overrides

- **Offset** — sets the object's **local offset inside** the deposit.
- **Override Rotation** — when **on**, lets you **adjust the object's rotation** while it is clamped.

Specific Object

When you assign a **Specific Object**, this deposit **only accepts** that object. Any other grabbed object entering the trigger **won't be affected**.

Events Tab

- **On Interactable Clamp** — fired when an object **gets clamped** in the deposit.
- **On Trigger Enter** — fired when **any collider** enters the Box (Is Trigger).
- **On Trigger Exit** — fired when **any collider** exits the Box.

Lerps

Lerp scripts automate one-off transitions (e.g., opening a drawer/door) without extra glue code: add the script and the system animates it. There are **4 lerp scripts**; their specifics live in **subpages**. Below are the **common** settings found in the **Lerps** tab of all of them.

Target

- **Affect other object**: when **enabled**, apply the lerp to **another object** (not the one hosting the script). Just reference it.

Behaviour

- **Duration**: total **duration** of the lerp.
- **Curve**: the **animation curve** the lerp will follow.
- **Wait Animation Finish**: when **on**, **blocks starting** a new lerp **until the current one finishes**.
- **Toggle With Original State**: when **on**, allows **toggling** between the object's **original state** and the **final state**. If you need to **change the original state at runtime**, call `ChangeOriginalState()`

Events

- **On Lerp Start**: invoked when the lerp **starts**.
- **On Complete**: invoked when the lerp **finishes**.

Position

Lerp dedicated to **position**.

- **What is going to affect:** chooses **local** or **global** space for the calculation.
- **New Pos:** the **Vector3** with the **target position** (local/global according to **What is going to affect**).

Rotation

Lerp dedicated to **rotation**.

- **What is going to affect:** chooses **local** or **global** rotation space.
- **Use Euler:** choose to affect **Euler Angles** (`.eulerAngles`) or **Quaternion** (`.rotation`) on the `Transform` (still respecting local/global from **What is going to affect**).
- **New Rot:** the **target rotation** (Euler or Quaternion per **Use Euler**) in **local** or **global** space per **What is going to affect**.

Scale

Lerp dedicated to **scale**.

- **What is going to affect:** chooses **local** or **global** scale space.
- **New Scale:** the **Vector3** with the **target scale** (local/global according to **What is going to affect**).

Transform

Lerp that applies a **full Transform state**.

- **What is going to affect:** applies **position**, **rotation**, and **scale** in **local** or **global** space.
- **Destination:** the `Transform` holding the **final state** (pos/rot/scale) to lerp to.

Settings

- **Change Position** — when on, the lerp **changes position**.
- **Change Rotation** — when on, the lerp **changes rotation**.
- **Change Scale** — when on, the lerp **changes scale**.

Curves

- **Override Position Curve** — use a specific **animation curve** for **position**.
- **Override Rotation Curve** — use a specific **animation curve** for **rotation**.
- **Override Scale Curve** — use a specific **animation curve** for **scale**.

UI Prefab

The **UI Prefab** contains two tabs: **Enabled Tab** and **Disabled Tab**. These tabs help you preview and organize what exists in that specific prefab.

For example, if the UI Prefab you are creating includes a **Background**, you can enable the background on the **Disabled Tab** and it will be carried over to the **Enabled Tab**.

In this UI Prefab you can reference the following objects:

Background

Reference the prefab **Image** that is responsible for the **Background**. Used by [Details](#) to enable/disable the background.

Name Text

Reference the prefab **Text** that will display the object's name in the hierarchy. Used by **prompts**, for example.

Image

Reference the prefab **Image** that will be shown on screen. This image is generally used by **While Interacting** (specifically by [Drag](#) and [Grab](#)).

Interaction Text

Reference the prefab **Text** that will display the interaction text. Used by **prompts**.

Slider

Reference the prefab **Slider**. Used by [Hold](#).

Details

Reference the **Text** and **Image** objects for the [inspection](#) details.

 You can also reference these objects via scripts. Get the **UI Prefab** with `GetComponent<UIPrefab>()` and modify these fields directly. For more information, see [Scripting](#).

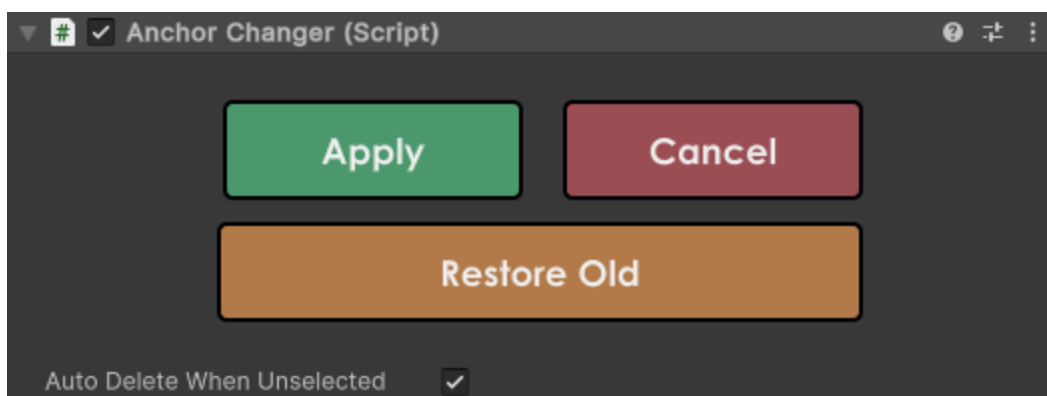
Setting new anchors

In several parts of the system you'll find options to **set anchors**. This page explains how they work.

There are two anchor definitions in the asset: the **Interactable** anchor and the **UI** anchor.

- The **Interactable** anchor mainly affects the system logic (e.g., where the object anchors **while dragging**, etc.).
- The **UI** can use its own **UI** anchor for display. For the UI, **the UI anchor has priority**: if both the Interactable anchor override and the UI anchor are present, the UI will use the **UI** anchor.

Whenever you click a **Set Anchor** button, a **temporary helper object** is created with the **Anchor Changer (Script)** attached to it.



This script is automatically associated with the new temporary anchor object:

- **Apply**: writes the `Temporary Anchor Transform` value to the field and deletes itself.
- **Cancel**: deletes the temporary anchor without applying changes.
- **Restore Old** (*visible only if the current anchor is different from `Vector3.zero`*): reverts to the previous position.
- **Auto Delete When Unselected**:
 - **On** — when you deselect the anchor object, it automatically applies the values and deletes the object.

- **Off** — you can switch to other objects without needing to apply/delete immediately.

How to set a new anchor: move the temporary anchor object to the desired position and click **Apply**.

If **Auto Delete When Unselected** is enabled, simply deselecting the object will also apply and remove it.

Scripting

The goal of our system is to **minimize the need for scripting** so that simple changes and effects can be done with a few clicks in the **Inspector**.

However, for more advanced needs (such as **runtime** changes) clicks alone aren't enough. In this section you'll find methods and variables that can be useful in code.

Getting Started

To begin, in any script where you want to interact with the system, **import the `FastStudios` namespace:**

```
using FastStudios;

public class YourClass : MonoBehaviour
{
    // You can now use this system's scripts in your code.
}
```

Interaction Manager

To access the manager anywhere:

```
InteractionManager manager = InteractionManager.singleton;
```

Useful Properties

Property	Description
interactionKey	Input key used to trigger an interaction . Changing this updates the key used by the whole system.
throwableKey	Input key used to throw a grabbed object (Grab).
ObtainedKeys	Runtime list containing all keys the player has obtained . Use for checks, debugging, or custom UI.

Useful Methods

Method	Descrição
DestroyKey	Destroys the given key object and removes it from <code>ObtainedKeys</code>
SetInteractionDistance	Sets the maximum interaction distance used by the system. Takes effect globally.
GetPlayerObject	Resolves and assigns the current player <code>GameObject</code> in the manager (based on your configured detection method) and returns it. If you pass <code>objectThatInteracted</code> , it's used as a shortcut.
MissingKeysString	Returns a human-readable string listing all missing keys for the given <code>Interactable</code> , respecting And/Or operators. Example format: <code>Key 1, Key 2 or Key 3, Key 4</code>
GetMissingKeys	Returns structured info about missing keys. Each tuple contains the specific <code>Key</code> (when a condition requires a specific key) or a name string (when the condition is name-based). One of the tuple items will be <code>null</code> .

Interactable

Useful Properties

Property	Description
interactMode	The interaction type used by this object (e.g., Drag, Grab, Hold, UnityEvent, Inspection). Not recommended to change at runtime.
maxInteractions	Maximum number of interactions allowed. Compared against <code>interactionTimes</code> if <code>interactionTimes</code> $\geq$ <code>maxInteractions</code> , interaction is blocked.
UseConditions	Enables evaluation of this Interactable's Conditions at runtime.
maxHoldInteractions	Maximum number of completed Hold interactions available. Decrements each time a Hold finishes. Requires <code>hasMaxHoldInteractions</code> to enforce a cap.
canSeeThrowTrajectory	(Grab only) Shows the throw trajectory preview when enabled.
DragLocked	(Drag only) Indicates if the object is currently locked . Read-only in practice (use <code>SetDragLocked()</code> to set)

Useful Methods

Method	Description
SetCanInteract	Enables/disables the object's ability to be interacted with (useful to block or re-enable interactions at runtime).
AddConditionConst	Adds a condition comparing a member to a constant value . Overloads accept either a component directly or a GameObject + component type. From the second condition onward, set <code>LogicalJoin</code> (And/Or).
AddConditionDynamic	Adds a condition comparing two members (left vs. right). Overloads accept components directly or GameObject + component type. From the second condition onward, set <code>LogicalJoin</code> (And/Or).
AddKeyCondition	Adds a key condition by key name or by a specific Key component. Use <code>LogicalJoin</code> from the second condition onward.
SetDragLocked	(Drag only) Locks/unlocks the drag object (doors, drawers, etc.).
GetSliderValue	(Drag with Slider) Returns/exposes the current Slider value.
DebugMessage	Prints a log message. Designed for Unity Events ; in code prefer <code>Debug.Log()</code> .

UI Prefab

Useful Properties

Property	Description
NameTmpText	Text object that displays the Interactable's name .
DetailsText	Text object used for the details text in inspection mode .
DetailsImage	Image object used for the details image in inspection mode .
Background	Image object used as the background for the inspection details.
Image	Image object shown during While Interacting (used by Drag and Grab).
Slider	Slider object used by Drag when the interaction exposes a slider value .

Useful Methods

Method	Description
SetInteractionText	Parses a string and replaces <code>{key}</code> / <code>{interactionKey}</code> and <code>{throwKey}</code> with the current keys. If an <code>Interactable</code> is provided, per-object overrides are respected; otherwise, keys come from <code>InteractionManager</code>.

Universals

To change [universal](#) values at runtime, use `UniversalsRuntime`.
It exposes the following functions:

- `SetBool`
- `SetInt`
- `SetFloat`
- `SetString`
- `SetEnumIndex`
- `SetColor`
- `SetVector2`
- `SetVector3`
- `SetVector2Int`
- `SetVector3Int`
- `SetQuaternion`
- `SetLayerMask`
- `SetKeyCode`

For each function you must provide:

1. **Which script** you're updating (only scripts that support universals, you can see them in the [Universals](#) Window),
2. The **property name**
3. The **new value**.

Example:

```

public void ChangeUniversals()
{
    // Interactable.maxInteractions -> 10
    UniversalsRuntime.SetInt<Interactable>
(nameof(Interactable.maxInteractions), 10);

    // Key.CanInteract -> true (using the property name as string)
    UniversalsRuntime.SetBool<Key>("CanInteract", true);

    // PositionLerp.Duration -> 3.5f
    UniversalsRuntime.SetFloat<PositionLerp>
(nameof(PositionLerp.Duration), 3.5f);

    // Key.KeyName -> "Universal Key"
    string KeyName = nameof(Key.KeyName);
    UniversalsRuntime.SetString<Key>(KeyName, "Universal Key");
}

```

Enums

InteractMode

```
public enum InteractMode
{
    UnityEvent = 1,
    Hold = 2,
    Grab = 3,
    Drag = 4,
    Inspection = 5
}
```

LogicalJoin

```
public enum LogicalJoin
{
    And,
    Or
}
```

ConditionOperator

```

public enum ConditionOperator
{
    [InspectorName("=")] Equal,
    [InspectorName("!=")] NotEqual,
    [InspectorName(">")] Greater,
    [InspectorName(">=")] GreaterOrEqual,
    [InspectorName("<")] Less,
    [InspectorName("<=")] LessOrEqual,

    [InspectorName("X =")] XEqual,
    [InspectorName("Y =")] YEqual,
    [InspectorName("Z =")] ZEqual,

    Contains,
    [InspectorName("Starts With")] StartsWith,
    [InspectorName("Ends With")] EndsWith,

    [InspectorName("Is True")] IsTrue, [InspectorName("Is False")]
IsFalse,
    [InspectorName("Is Null")] IsNull, [InspectorName("Not Null")]
NotNull
}

```